



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Uriarte Ortiz Enrique Yahir

N° de Cuenta: 318234757

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: Sábado 12 de Octubre del 2024

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios.

- 1) Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del cofre de su coche y al abrir y cerrar el cofre ilumine en esa dirección.

Para comenzar esta actividad primero copie la parte de código de la practica 5 para asignar el movimiento del cofre del coche, la estructura en los archivos "Window.h" y "Window.cpp" es la misma, hasta la asignación de teclas para el moviento.

```
GLfloat getmuevex() { return muevex; }  
GLfloat getarticulacion1() { return articulacion1; }
```

Fig. 1. Definición movimiento para el cofre en "Window.h".

```
GLfloat muevex;  
GLfloat articulacion1;
```

Fig. 2. Definición movimiento para el cofre en "Window.h".

```
~Window::Window(GLint windowHeight, GLint windowHeight){  
    width = windowHeight;  
    height = windowHeight;  
    muevex = 2.0f;  
    articulacion1 = 0.0f;  
    lampLightOn = true;  
    newLightOn = true;  
    direccionCoche = 0; // Inicialización  
    for (size_t i = 0; i < 1024; i++){keys[i] = 0;}  
}
```

Fig. 3. Definición movimiento para el cofre en "Window.cpp".

```
if (key == GLFW_KEY_Z){//Abrir el cofre.  
    if (theWindow->articulacion1 > 46.0){}  
    else { theWindow->articulacion1 += 2.0;}}  
if (key == GLFW_KEY_X){//Cerrar el cofre.  
    if (theWindow->articulacion1 < 1.0){}  
    else {theWindow->articulacion1 -= 2.0;}}
```

Fig. 4. Definición movimiento para el cofre en "Window.cpp".

Después en el código *main* declaro la nueva spotlight.

```
//Luz de cofre.  
spotLights[0] = SpotLight(1.0f,0.0f,0.0f, 4.0f,1.0f, 0.0f,0.0f,0.0f, -1.0f,0.0f,0.0f, 1.0f,0.1f,0.1f, 25.0f);  
spotLightCount++;
```

Fig. 5. Definición de spotlight para el cofre en "Window.cpp".

Y por último en el bucle de ejecución, después de colocar el cofre, declaro las coordenadas de este a partir de la jerarquía de coche, así como la dirección a la que se dirige la luz en ese punto, hacia -X, después se crea una matriz de rotación para que el foco rote en función del ángulo del cofre, definida alrededor de -Z.

Después aplico la rotación al vector de dirección de la luz LuzCofre_D, a partir de la rotación RotCofre, definiendo que este rotando solo la dirección sin afectar su posición, luego normalizo la dirección de la luz para asegurar que su longitud sea 1, esto es para evitar problemas en los cálculos de iluminación.

Finalmente, se actualiza el spotlight con la nueva posición y dirección de la luz, llamado SetFlash().

```
// Ejercicio 1. Spotlight en el cofre de su coche, al abrir y cerrar el cofre ilumine en esa dirección.
glm::vec3 LuzCofre_P = glm::vec3(Cofre[3]);
glm::vec3 LuzCofre_D = glm::vec3(-1.0f, 0.0f, 0.0f);
glm::mat4 RotCofre = glm::rotate(glm::mat4(1.0f), glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, -1.0f));
LuzCofre_D = glm::vec3(RotCofre * glm::vec3(LuzCofre_D, 0.0f));
LuzCofre_D = glm::normalize(LuzCofre_D); //Normalizar la dirección
spotlights[0].setFlash(LuzCofre_P, LuzCofre_D);
```

Fig. 6. Asignación de luz en el cofre en “Window.cpp”.

- 2) Agregar luz de tipo spotlight para el coche de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight hacia atrás. Son dos spotlights diferentes que se prenderán y apagaran de acuerdo con alguna bandera asignada por ustedes.

Para esta segunda actividad primero declare los Spotlight que utilizare, el primero manteniendo el color azul de la partica anterior pero el segundo de color morado.

```
//Foco Frontal.
spotlights[1] = SpotLight(0.0f,0.0f,1.0f, 4.0f,1.0f, 0.0f,0.0f,0.0f, -1.0f,0.0f,0.0f, 1.0f,0.1f,0.05f, 8.0f);
spotLightCount++;

//Foco Trasero.
spotlights[2] = SpotLight(1.0f,0.0f,1.0f, 4.0f,1.0f, 0.0f,0.0f,0.0f, 1.0f,0.0f,0.0f, 1.0f,0.1f,0.05f, 8.0f);
spotLightCount++;
```

Fig. 7. Definición de spotlight para la parte frontal y tracera.

Después en los archivos “SpotLight.h” y “SpotLight.cpp” declare una nueva función para ubicar con precisión en cada eje donde se ubicaban las luces.

```
void SetFlash(glm::vec3 pos, glm::vec3 dir);
void SetPos(glm::vec3 pos);
void SetPos_Especific(GGLfloat x, GGLfloat y, GGLfloat z);
~SpotLight();
```

Fig. 8. Definición de función para la ubicación de los SpotLights en “SpotLight.h”.

```
void SpotLight::setFlash(glm::vec3 pos, glm::vec3 dir){position = pos;direction = dir;}
void SpotLight::SetPos(glm::vec3 pos){position = pos;}
void SpotLight::SetPos_Especific(GGLfloat x, GGLfloat y, GGLfloat z) { position.x = x; position.y = y; position.z = z; }
```

Fig. 9. Función para la ubicación de los SpotLights en “SpotLight.cpp”.

Para que el archivo main sepa en que direccion va el coche añadi una nueva variable llamada dirección coche, si el valor era 1 entonces el carro se estaba moviendo hacia enfrente, pero si el valor era de -1 entonces el carro se estaba moviendo hacia atrás. Esta variable la declare en “Window.h” y “Window.cpp”. Además cuando se sueltan las teclas Y o U, las teclas del movimiento hacia atrás y hacia delante, se restablece la variable direccionCoche a 0, ya que el coche ya no se está moviendo y ninguna luz debe prenderse.

```
GLfloat getmuevex() { return muevex; }
GLfloat getarticulacion1() { return articulacion1; }
int getDireccionCoche() { return direccionCoche; }
```

Fig. 10. Declaración de dirección del carro en “Window.h”.

```
GLfloat muevex;
GLfloat articulacion1;
int direccionCoche;
```

Fig. 11. Declaración de dirección del carro en “Window.h”.

```

Window::Window(GLint windowWidth, GLint windowHeight){
    width = windowWidth;
    height = windowHeight;
    muevex = 2.0f;
    articulacion1 = 0.0f;
    lampLightOn = true;
    newLightOn = true;
    direccionCoche = 0; // Inicialización
    for (size_t i = 0; i < 1024; i++){keys[i] = 0;}
}

```

Fig. 12. Declaración de dirección del carro en "Window.cpp".

```

if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS){glfwSetWindowShouldClose(window, GL_TRUE);}

if (key == GLFW_KEY_Y){
    theWindow-> muevex += 1.0;
    theWindow->direccionCoche = -1; // Moviendo hacia atrás
}

if (key == GLFW_KEY_U){
    theWindow-> muevex -= 1.0;
    theWindow->direccionCoche = 1; // Moviendo hacia adelante
}

// Resetear dirección cuando se sueltan las teclas
if ((key == GLFW_KEY_Y || key == GLFW_KEY_U) && action == GLFW_RELEASE){
    theWindow->direccionCoche = 0;
}

```

Fig. 13. Declaración de dirección del carro en "Window.cpp".

Después en el código main declare dentro del bucle de ejecución la variable de dirección del coche para mandar la información de movimiento del coche.

```

int direccionCoche = mainWindow.getDireccionCoche();

```

Fig. 14. Declaración de dirección del carro en archivo main.

Una vez declarado el guardado de posiciones a cada jerarquía del carro, declare las instrucciones para las luces, primero se guarda la posición de origen, a los 2 spotlights les asigno su ubicación con respecto a la ubicación de la jerarquía original, una al frente y la otra en la parte trasera, la de color azul y la de color morada respectivamente, después declaro una serie de ifs para declarar el encendido de las luces, en el primer if se declara que si el coche está avanzando se enciende el foco delantero (spotLights[1]) con dirección a -X y más ajustado en posición, pero si el coche está retrocediendo se enciende el foco trasero (spotLights[2]), en último caso si la dirección no tiene ninguno de estos valores quiere decir que el coche no se está moviendo, por lo tanto ambos focos se apagan.

```

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(mainWindow.getMuevex(), 0.7f, 2.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.05f, 0.05f));
Cofre = model;
Llanta01 = model;
Llanta02 = model;
Llanta03 = model;
Llanta04 = model;

glm::vec3 posicionCoche = glm::vec3(model[3]);
spotLights[1].SetPos_Especific(posicionCoche.x, posicionCoche.y + 0.5f, posicionCoche.z + 2.0f);
spotLights[2].SetPos_Especific(posicionCoche.x, posicionCoche.y + 0.5f, posicionCoche.z - 2.0f);

// Ejercicio2. Cuando el coche avanza se enciende un spotlight hacia adelante, al retroceder enciende un spotlight hacia atrás
if (direccionCoche == 1) { // Encender foco frontal, apagar foco trasero
    spotLights[1].SetFlash(glm::vec3(posicionCoche.x - 4.0f, posicionCoche.y - 0.15f, posicionCoche.z + 1.5f),
        glm::vec3(-1.0f, 0.0f, 0.0f));
}
else if (direccionCoche == -1) { // Apagar foco frontal, encender foco trasero
    spotLights[2].SetFlash(glm::vec3(posicionCoche.x + 4.0f, posicionCoche.y + -0.15f, posicionCoche.z - 1.5f),
        glm::vec3(1.0f, 0.0f, 0.0f));
}
else { // Apagar ambos focos cuando no hay movimiento
    spotLights[1].SetFlash(posicionCoche, glm::vec3(0.0f));
    spotLights[2].SetFlash(posicionCoche, glm::vec3(0.0f));
}

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_Cuerpo.RenderModel();

```

Fig. 15. Implementación de encendido y apagado de luces del coche.

- 3) Agregar otra luz de tipo puntual ligada a un modelo elegido por ustedes (no lámpara) y que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara como la luz de este modelo (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual). Para comenzar esta actividad analice que elemento podría escoger para este ejercicio considerando que debía emitir luz, al final escogí un celular, ya que la pantalla de este da luz al momento de encenderlo, después busque el modelo y escogí el siguiente.

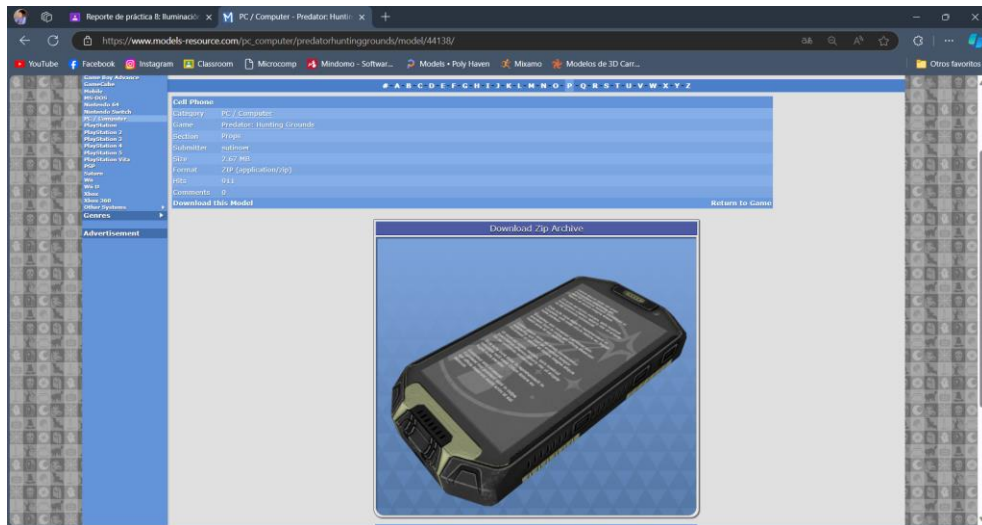


Fig. 16. Modelo de celular escogido para la actividad.

Después lo ajuste en 3ds Max y lo guarde en la capeta de modelos en el proyecto, también guarde las texturas de este en la carpeta de texturas del proyecto.

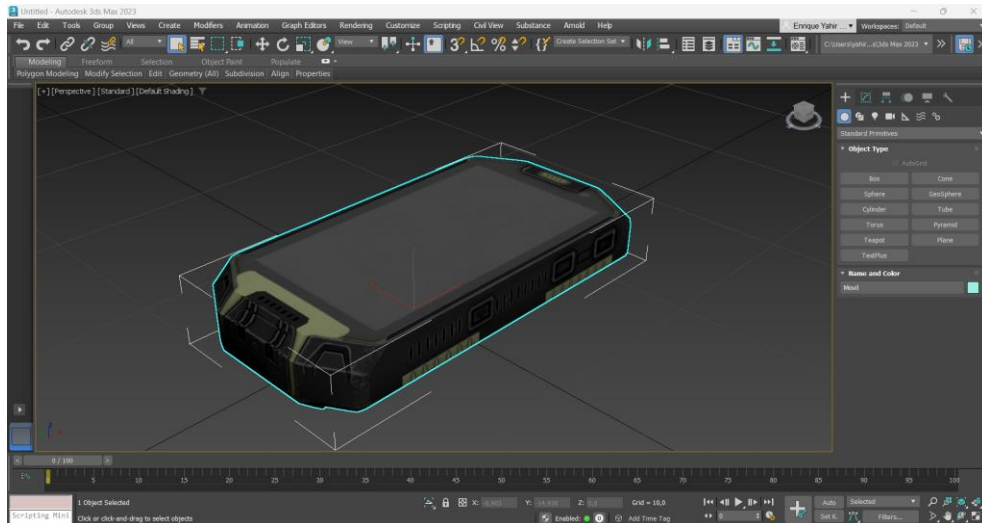


Fig. 17. Ajuste de posición de pivote del celular en 3ds Max.

Después como en el ejercicio de esta práctica, cree una nueva luz puntual, asignándola tanto en el archivo “Window.h” y “Window.cpp” que recibía un valor booleano para encenderse. Dentro del archivo “Window.cpp” asigne la tecla K para

el cambio de arreglos durante la ejecución, se invierte el valor de newLightOn, true a false y viceversa.

```
bool getLampLightState() { return lampLightOn; }
bool getNewLightState() { return newLightOn; }
```

Fig. 18. Declaración de encendido de la luz puntual en "Window.h".

```
bool lampLightOn;
bool newLightOn;
```

Fig. 19. Declaración de encendido de la luz puntual en "Window.h".

```
Window::Window(GLint windowWidth, GLint windowHeight){
    width = windowWidth;
    height = windowHeight;
    muevex = 2.0f;
    articulacionl = 0.0f;
    lampLightOn = true;
    newLightOn = true;
    direccionCoche = 0; // Inicialización
    for (size_t i = 0; i < 1024; i++){keys[i] = 0;}
}
```

Fig. 18. Declaración de encendido de la luz puntual en "Window.cpp".

```
if (key == GLFW_KEY_L && action == GLFW_PRESS) {theWindow->lampLightOn = !theWindow->lampLightOn;}
if (key == GLFW_KEY_K && action == GLFW_PRESS) {theWindow->newLightOn = !theWindow->newLightOn;}
```

Fig. 19. Declaración de encendido de la luz puntual en "Window.cpp".

En el archivo main declare el modelo del celular y la ubicación de este dentro del proyecto. Después declare la nueva luz puntual dentro del código, teniendo los mismos valores que la luz puntual para la lampara que tenía previamente.

```
Model Carro_llantas;
Model Carro_Cuerpo;
Model Carro_Cofre;
Model Lampara;
Model Movil;

Skybox skybox;
```

Fig. 20. Declaración de modelo en archivo main.

```
Lampara = Model();    Lampara.LoadModel("Models/Lampara.obj");
Movil = Model();      Movil.LoadModel("Models/Movil.obj");
```

Fig. 21. Declaración de la ubicación del modelo en archivo main.

```
pointLights[1] = PointLight( //Luz de Celular.
    1.0f, 1.0f, 1.0f,
    1.8f, 1.0f,
    0.0f, 0.0f, 0.0f,
    1.0f, 0.4f, 0.2f);
pointLightCount++;
```

Fig. 22. Declaración de pointlight en archivo main.

Después declare un contador para rastrear el numero de luces puntuales activas y un arreglo para almacenar hasta dos luces puntuales activas (una para cada modelo), esta implementación la considera para no sobrescribir las luces puntuales cada vez, de modo que no borre la luz de la lampara cuando configuro la del celular.

```
int direccionCoche = mainWindow.getDireccionCoche();

unsigned int activeLightCount = 0;
PointLight activeLights[2];
```

Fig. 23. Declaración de contador para saber las luces puntuales activas.

Luego asigno los modelos con sus ubicaciones y sus escalas, después de cada mandar a llamar a cada modelo, asigno la posición de la lámpara a partir de `modelaux`, ajusto la posición de la luz puntual asociada a la lámpara, a una ubicación desplazada de la lámpara, y declaro el if de ejecución, si la luz de la lámpara está activada, verificado con `getLampLightState()`, se agrega `pointLights[0]` al arreglo `activeLights`, y se incrementa el contador `activeLightCount`. Esto mismo ocurre en la asignación del móvil, pero con sus propias variables.

La razón de esta asignación es configurar todas las luces activas de una sola vez, pasando el array de luces activas y el número total de luces activas, asegurando con esto que ambas luces se activen o desactiven correctamente y de manera independiente.

```
// Ejercicio 3. Dos luces puntuales cada una a un modelo diferente, se prenden y apagan de forma independiente.
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.95f, -10.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.05f, 0.05f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara.RenderModel();
glm::vec3 lamparaPos = glm::vec3(modelaux[3]);
pointLights[0].SetPosition(lamparaPos.x + 1.45f, lamparaPos.y + 2.62f, lamparaPos.z);
if (mainWindow.getLampLightState()) {activeLights[activeLightCount++] = pointLights[0];}

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 15.0f));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux2 = model;
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Movil.RenderModel();
glm::vec3 MovilPos = glm::vec3(modelaux2[3]);
pointLights[1].SetPosition(MovilPos.x + 0.0f, MovilPos.y + 1.0f, MovilPos.z);
if (mainWindow.getNewLightState()) {activeLights[activeLightCount++] = pointLights[1];}

shaderList[0].SetPointLights(activeLights, activeLightCount);
```

Fig. 24. Declaración de luces puntuales y su activación.



Fig. 25. Ejecución del ejercicio.

Como evidencia para la práctica, realice un video donde se muestra la ejecución, el archivo se llama P08-318234757_Ejercicios.mp4, adjunto el enlace compartido: <https://drive.google.com/file/d/1haREr2VQwoq87dYrGss7MEY9CyegQCOi/view?usp=sharing>

2. *Problemas presentados.*

A diferencia de prácticas anteriores, en esta si presente complicaciones en la primer y última actividad. En la primera actividad no entendí muy bien la ubicación donde debía estar la luz, gracias a la asesoría del profesor entendí donde debía ir esta y el seguimiento que debía tener con el cofre del coche, pero aun así fue complicado asignar el seguimiento de la rotación, al final intente asignar la dirección de la luz con la posición de rotación para mantenerla constante, lo que soluciono el movimiento. Para la segunda actividad fue un poco mas sencillo el asignar con ifs el encendido de las luces con el if, sin embargo, la solución de mandar un valor de acuerdo con el sentido hacia el que se movía el coche se me ocurrió tras tener problemas al asignar el encendido de las luces correspondientes, lo demás fue más sencillo al solo asignar cada luz a cada situación. Y en el caso de la última actividad en un inicio intente usar lo mismo que la practica anterior de asignar las luces a cada modelo, sin embargo, cuando configuraba la segunda luz, la del móvil, esta se sobrescribía con la configuración de la luz de la lampara, provocando que la luz de esta se mantuviera encendida y no la pudiera controlar, pero la del celular si respondía a la tecla, para solucionar esto debía mandar el encendido de cada luz de manera independiente, cree un arreglo donde se almacenaría en tiempo de ejecución las luces puntuales que debían encenderse, así mandaría las luces activas de una sola vez, pasando por un arreglo donde se almacenaban las luces activas y el número total de luces activas, asegurando con esto que ambas luces se activen o desactiven correctamente y de manera independiente.

3. *Conclusión:*

Tras realizar los ejercicios de la practica me parece que la complejidad fue mayor a prácticas anteriores, la implementación de iluminación en los modelos en diversas formas como seguir la rotación de un modelo en específico, limitar las condiciones de encendido de las luces o controlar diferentes luces al mismo tiempo de manera independiente cada una me pareció interesante de implementar. Al final consideró que esta práctica cumplió su objetivo de ampliar las formas de usar las luces, dándole un poco más de realismo a estos, siendo esto a mi parecer una buena implementación para el desarrollo del proyecto final.

4. *Bibliografía.*

- *PC / computer - Predator: Hunting Grounds - cell phone. The Models Resource.* https://www.models-resource.com/pc_computer/predatorhuntinggrounds/model/44138/.
Accedido en octubre 8, 2024.